

Slide 1 — Título e Identificação

Bom dia a todos, meu nome é George Gomes e esta é a defesa da minha dissertação de mestrado, intitulada “Explicando Decisões de LLMs via Otimização Inversa”.

Este trabalho investiga se modelos de linguagem de grande escala — os LLMs — possuem um processo de tomada de decisão que pode ser capturado matematicamente. A ideia central é usar técnicas de otimização inversa para extrair uma métrica que explique como o LLM decide.

Vou estruturar a apresentação em 9 partes: motivação, design experimental, os resultados das 4 fases do experimento, análises de viés e sensibilidade, validações adicionais, análise estatística, e conclusões.

O ponto de partida deste trabalho é uma observação simples mas importante: LLMs tomam decisões o tempo todo — classificam textos, respondem perguntas, fazem julgamentos — mas não explicam qual critério estão usando internamente.

A pergunta central que nos guia é: “LLMs possuem um processo de decisão estável e aprendível?” Se a resposta for sim, então podemos extrair uma métrica matemática W que funcione como uma “explicação” das decisões do modelo. Isso tem implicações diretas para explainability, para construção de confiança em sistemas de IA, e para auditoria de decisões automatizadas.

A relevância prática é clara: se eu consigo descrever matematicamente como o LLM decide, posso prever, auditar e eventualmente corrigir essas decisões.

Slide 3 — Hipóteses

Formulamos cinco hipóteses que guiam todo o trabalho experimental.

H1 afirma que o LLM mantém o mesmo critério decisório quando enfrenta problemas com geometrias diferentes. H2 diz que fornecer exemplos rotulados no prompt aumenta a concordância entre o LLM e a métrica aprendida. H3 inverte o papel: pergunta se o LLM consegue aprender o critério de um perito externo via few-shot learning. H4 é particularmente interessante — postulamos que exemplos próximos à fronteira de decisão seriam mais informativos que exemplos fáceis. E H5 testa se tudo isso é reproduzível entre diferentes execuções com sementes aleatórias distintas.

Já adianto que nem todas foram confirmadas, e a refutação de H4 é um dos achados mais relevantes do trabalho.

Slide 4 — Abordagem (Otimização Inversa)

A abordagem é conceitualmente elegante: dado que o LLM tomou decisões observáveis, tentamos descobrir qual função objetivo tornaria essas decisões ótimas. Isso é exatamente o que a otimização inversa faz.

A métrica que buscamos é uma distância de Mahalanobis diagonal, onde cada peso w_i indica a importância relativa de cada feature. Escolhemos a forma diagonal deliberadamente: com $O(d)$ parâmetros em vez de $O(d^2)$, ganhamos interpretabilidade e tratabilidade computacional. Cada w_i tem significado direto.

A classificação é do tipo nearest-centroid: cada ponto é atribuído ao centroide mais próximo segundo a métrica W . Isso é simples, mas suficiente para capturar critérios anisotrópicos — ou seja, critérios que pesam as dimensões de forma diferente.

Slide 5 — Dados Sintéticos 2D

Usamos dados sintéticos bidimensionais gerados com a função `make_blobs` do `scikit-learn`. São 4 problemas distintos com distribuições Gaussianas.

A escolha por dados 2D é proposital: permite visualização direta das fronteiras de decisão, dos pontos classificados corretamente e incorretamente, e da geometria aprendida. Cada problema tem centroides e espalhamento diferentes, criando geometrias distintas para testar a generalização da métrica.

No gráfico à direita vocês podem ver os scatter plots dos problemas lado a lado. As cores representam as classes verdadeiras do ground truth.

O Problema A é o problema de treino. Tem 150 pontos divididos em 2 classes. É neste problema que o LLM faz suas classificações em zero-shot — sem nenhum exemplo — e a partir dessas 150 decisões, nossos algoritmos de otimização inversa inferem a métrica W que melhor explica as escolhas do LLM. Os parâmetros dos centroides e do `cluster_std` foram escolhidos para criar classes com alguma sobreposição, mas não excessiva. Isso é importante: se os clusters fossem perfeitamente separados, qualquer métrica funcionaria; a sobreposição é o que nos permite distinguir métricas diferentes.

Slide 7 — Problema B

O Problema B tem 100 pontos e centroides deslocados verticalmente em relação ao Problema A. A geometria é diferente, mas a tarefa é a mesma: classificação binária em 2D.

Este é o primeiro teste de consistência: aplicamos a métrica aprendida no Problema A e verificamos se o LLM, classificando pontos do Problema B, concorda com as previsões dessa métrica transferida. Se concordar, temos evidência de que o critério decisório do LLM é estável e transferível entre problemas.

O Problema C é semelhante ao B, mas com distorção geométrica mais severa — 100 pontos com uma configuração mais desafiadora. Funciona como um segundo nível de teste: se a métrica generaliza para o Problema B mas não para o C, sabemos que a consistência tem limites e depende de quão diferente é a geometria.

A combinação B+C nos dá uma visão gradual: consistência moderada a difícil.

O Problema D é fundamentalmente diferente dos anteriores. Aqui, temos um expert externo com uma métrica W conhecida — especificamente, $W = [0.3, 1.5]$, o que significa que a segunda dimensão, x_2 , é 5 vezes mais importante que x_1 na classificação.

O expert rotula os 150 pontos do Problema D usando essa métrica. O papel do LLM aqui é invertido: em vez de ser o “sujeito” cujas decisões analisamos, o LLM é o “aprendiz” que deve aprender o critério do expert a partir de exemplos. No gráfico à direita vocês podem ver o Problema D com a fronteira de decisão do expert claramente desenhada.

Slide 10 — Configurações de Execução

Um dos pilares do trabalho é o design de variabilidade. Precisamos garantir que os resultados não são artefatos de uma configuração específica.

Usamos 3 sementes aleatórias diferentes — 42, 123 e 7 — que controlam a geração dos dados. Para cada configuração, fazemos 3 repetições. A temperatura do modelo está fixa em 0.0 para minimizar a estocasticidade — queremos isolar o critério decisório, não o ruído da amostragem do modelo. Usamos o gpt-4o-mini via API da OpenAI.

Para detectar viés semântico, testamos 4 pares de nomes de classe: A/B que é neutro, 0/1 que é numérico, Positivo/Negativo e Azul/Vermelho que são semânticos. A ideia é verificar se o nome da classe influencia a decisão do LLM — e, como veremos, influencia significativamente.

Slide 11 — Perceptron Estruturado

O primeiro algoritmo é um Perceptron Estruturado com relaxação de margem. A ideia é buscar iterativamente os pesos W que maximizem a separação entre as classes segundo a métrica de Mahalanobis diagonal.

O procedimento usa busca binária sobre o parâmetro γ , que é a margem mínima desejada. Para cada valor de γ , treinamos o perceptron até convergência: quando a métrica erra — ou seja, atribui um ponto à classe errada segundo o LLM — atualizamos os pesos na direção que corrige o erro. Os pesos são projetados para não-negatividade após cada atualização.

O resultado é o W com o maior γ que ainda permite convergência. Quanto maior o γ , mais robusta é a separação — mas se as decisões do LLM forem inconsistentes, γ será pequeno ou zero.

Slide 12 — Mínimos Quadrados (NNLS)

O segundo algoritmo aborda o problema de forma diferente: formula as restrições de classificação correta como um sistema de desigualdades lineares e resolve via Non-Negative Least Squares.

Para cada ponto classificado como classe k pelo LLM, escrevemos a restrição de que a distância à classe correta deve ser menor que a distância a qualquer outra classe. Isso gera um sistema $Aw \leq b$ que resolvemos minimizando o resíduo quadrático com a restrição $w \geq 0$.

A grande vantagem é que a solução não é iterativa — é computada diretamente. E o fato de termos dois algoritmos independentes nos dá uma validação cruzada: se ambos convergem para um W similar, o resultado é mais confiável.

Slide 13 — Programação Linear (LP)

O terceiro algoritmo é uma formulação de programação linear que maximiza a margem mínima sobre todas as observações. A ideia é encontrar o W que, sujeito a classificar todos os pontos corretamente segundo a métrica, maximiza a menor margem individual.

A normalização de que a soma dos pesos é igual a 1 evita a solução trivial de aumentar todos os pesos indefinidamente. A solução é via o solver HiGHS do scipy.

Quando γ ótimo é zero, significa que não existe nenhuma métrica diagonal que separe perfeitamente os dados segundo as decisões do LLM. Na prática, isso aconteceu em vários cenários da nossa execução, o que indica que o LLM não é perfeitamente consistente com nenhuma métrica de Mahalanobis diagonal.

Vamos agora aos resultados. Na Fase A, o LLM classifica 150 pontos do Problema A em zero-shot. O prompt é extremamente simples: apresenta as coordenadas do ponto e pede que o modelo classifique como A ou B.

A partir dessas 150 decisões, cada um dos 3 algoritmos infere uma métrica W . É importante frisar que chamamos isso de “ W estimado” ou “ W inferido”, não “ W aprendido pelo LLM” — o aprendizado é nosso, via otimização inversa, sobre as decisões observadas do LLM.

O W resultante representa o critério decisório implícito que melhor explica as classificações feitas pelo modelo.

Slide 15 — Fase A — W e Fidelidade

Aqui temos os resultados concretos. Para $\text{seed}=42$ com classes A/B, o Perceptron estimou W unitário como $[1.0, 0.0]$, ou seja, só a primeira dimensão importa. O NNLS estimou $[0.20, 0.80]$, priorizando a segunda dimensão. O LP ficou em $[0.71, 0.71]$, próximo da euclidiana.

A fidelidade — ou seja, a porcentagem de decisões do LLM que a métrica reproduz no próprio Problema A — ficou entre 72.7% e 80.7%. Isso não é perfeito, e sinaliza que o LLM não é totalmente consistente com nenhuma métrica diagonal simples.

Um detalhe importante: a acurácia do LLM versus o ground truth é apenas 30.7%. Isso parece ruim, mas lembrem que estamos medindo se a métrica reproduz o LLM, não se o LLM acerta. Um LLM pode ter um critério estável que simplesmente discorda do ground truth.

Este é um resultado preocupante para H5. Quando olhamos a distribuição de W estimado entre as 3 seeds, vemos instabilidade significativa. Na seed 42, o Perceptron estimou W apontando na direção de x_1 ; nas seeds 123 e 7, apontou na direção de x_2 — praticamente ortogonais.

A similaridade cosseno média entre seeds é apenas 0.53, e o coeficiente de variação da razão w_1/w_2 é de 258.9% — extremamente instável. Isso sugere que o critério decisório zero-shot do LLM pode não ser estável o suficiente para ser capturado de forma confiável.

No entanto, isso pode refletir que o LLM, sem orientação, simplesmente não tem um critério forte e usa heurísticas que variam com a disposição espacial dos dados.

Slide 17 — Comparação de Algoritmos

Comparando os 3 algoritmos, vemos que NNLS e LP são mais concordantes entre si, com similaridade cosseno de 0.87. O Perceptron diverge mais em certas seeds, especialmente quando os dados não são linearmente separáveis sob nenhuma métrica diagonal — nesses casos, ele atinge soluções de “melhor esforço” que podem ser degeneradas.

A fidelidade média confirma que o NNLS é ligeiramente superior: 82.4% contra 76% do Perceptron e 75.2% do LP. O LP frequentemente retorna a solução isotrópica $[0.5, 0.5]$ quando o problema é infeasible, o que é uma forma de “fallback” para a métrica euclidiana.

A divergência entre algoritmos não invalida a abordagem — pelo contrário, mostra que diferentes formulações capturam aspectos diferentes da inconsistência do LLM.

As Fases B e C testam a pergunta central de H1: a métrica aprendida na Fase A generaliza para problemas com geometria diferente?

O procedimento é direto: pegamos o W estimado no Problema A, calculamos centróides locais nos Problemas B e C, e comparamos as previsões da métrica com as classificações do LLM nesses novos problemas.

Usamos três métricas de concordância: a consistência bruta em porcentagem, o Cohen's Kappa que ajusta pelo acaso, e o F1-Score. O Kappa é particularmente importante porque leva em conta a distribuição das classes — um classificador que sempre diz “A” pode ter alta consistência bruta com outro que também tende a dizer “A”, mas o Kappa revelará que a concordância é espúria.

Slide 19 — Fases B/C Resultados

Os resultados aqui são muito reveladores. Em zero-shot, a consistência é fraca: 64% no Problema B e 63% no C, com Kappa de apenas 0.14. Isso significa que, sem exemplos, o LLM e a métrica concordam pouco mais do que o acaso.

Mas o salto com few-shot é dramático: com apenas 5 exemplos rotulados pela métrica, a consistência sobe para 91.7% e 92.7%, com Kappa acima de 0.80 — concordância substancial segundo os critérios de Landis e Koch. Com 10 exemplos, chegamos a 96.3% e Kappa de 0.93.

A saturação ocorre por volta de 10-20 exemplos; adicionar mais não melhora significativamente. Isso mostra que poucos exemplos bem escolhidos são suficientes para “calibrar” o LLM com a métrica.

Vamos à interpretação. H1, sobre consistência, é parcialmente confirmada: o LLM mantém consistência elevada com a métrica quando recebe exemplos, mas não em zero-shot. Isso sugere que o LLM tem a *capacidade* de seguir um critério consistente, mas precisa de orientação para ativá-lo.

H2 é claramente confirmada: exemplos rotulados pela métrica amplificam enormemente a concordância. O salto de zero-shot para 5-shot é de aproximadamente 30 pontos percentuais — um efeito muito grande.

Uma limitação importante é a fidelidade baixa na Fase A: se a métrica já não reproduz perfeitamente o LLM no Problema A, a base para transferência é frágil. Isso levanta uma questão sobre circularidade no few-shot: os exemplos são rotulados pela própria métrica, então estamos essencialmente “ensinando” o LLM a concordar conosco.

Slide 21 — Viés de Ordem das Classes

Uma análise de sensibilidade crucial é o viés de ordem das classes. Testamos se trocar “A ou B” por “B ou A” no prompt — ou seja, simplesmente inverter a ordem em que as classes são apresentadas — muda as decisões do LLM.

Os resultados mostram que sim, existe viés posicional. O exemplo mais dramático é o par A/B versus B/A: a consistência no Problema B salta de 79.5% para 94.9% quando invertemos a ordem. Isso é uma diferença de 15.4 pontos percentuais, causada apenas pela ordem das palavras no prompt.

Esse achado é relevante porque mostra que a métrica W inferida pode ser parcialmente um artefato da formulação do prompt, não apenas do critério decisório intrínseco do LLM. É um confundidor que precisa ser reportado e controlado.

Outro teste de sensibilidade: o que acontece quando mudamos os nomes das coordenadas de “ x_1 , x_2 ” para “altura, peso” ou “feature_1, feature_2”?

Nomes como “altura” e “peso” são semanticamente carregados — o LLM pode ter conhecimento prévio sobre a relação entre essas variáveis, o que influenciaria seus pesos implícitos. Por exemplo, pode “saber” que peso é mais discriminativo para certas classificações.

Os resultados confirmam que o LLM é sensível aos nomes das features. O W estimado muda dependendo dos nomes usados, o que é mais uma evidência de que o critério decisório do LLM é influenciado pelo contexto semântico do prompt, não apenas pelos dados numéricos.

Slide 23 — Variantes de Prompt (Design)

Para quantificar mais formalmente a sensibilidade ao prompt, testamos 4 variantes de template. A variante default é o nosso template baseline. A variante geometric adiciona contexto espacial explícito, como “considere a distância entre os pontos”. A variante chain-of-thought pede que o LLM raciocine passo a passo antes de responder. E a variante tabular apresenta as coordenadas como uma tabela markdown em vez de texto corrido.

Cada variante executa as Fases A, B e C completas em todas as seeds, o que nos dá uma comparação controlada. A análise mais importante é o scatter de W aprendidos: se os pontos se agrupam por variante e não por seed, o prompt é um confundidor forte.

Slide 24 — Variantes de Prompt (Resultados)

Os resultados mostram sensibilidade moderada ao template do prompt. O W estimado varia entre variantes, com a variante chain-of-thought e a geometric tendendo a alterar a distribuição de decisões do LLM em relação ao baseline.

A implicação é séria: se a métrica muda com o prompt, até que ponto estamos medindo algo intrínseco ao LLM versus algo que depende de como formulamos a pergunta? Isso é análogo a um instrumento de medição que produz resultados diferentes dependendo de como você segura ele.

No entanto, a consistência few-shot se mantém alta em todas as variantes, o que sugere que a capacidade do LLM de seguir um critério com orientação é mais robusta que o critério zero-shot em si.

Agora entramos na Fase D, que é conceitualmente diferente de tudo anterior. Aqui o papel se inverte: o LLM não é o sujeito de análise, mas o aprendiz.

Temos um expert externo com uma métrica W conhecida — $[0.3, 1.5]$, que dá peso 5 vezes maior à segunda dimensão. O expert rotula os pontos do Problema D e fornece exemplos ao LLM via few-shot. A pergunta é simples: com quantos exemplos o LLM consegue reproduzir as classificações do expert? E importa como esses exemplos são escolhidos?

No gráfico do Problema D com a fronteira do expert, vocês podem ver que a fronteira é inclinada — refletindo a anisotropia do W . Pontos longe da fronteira são “fáceis” e pontos próximos são “difíceis”.

Testamos 4 estratégias para escolher quais exemplos fornecer ao LLM. A estratégia Easy seleciona pontos longe da fronteira de decisão — são os pontos que o expert classifica com alta confiança. A estratégia Hard seleciona pontos próximos à fronteira — são os ambíguos, que em princípio carregam mais informação sobre onde está a fronteira. A estratégia Mixed combina 50% de cada. E Random é o baseline sem critério.

No gráfico à direita, vocês podem ver a distribuição espacial: os pontos Easy estão nas extremidades dos clusters, e os Hard estão aglomerados na região de transição entre as classes.

A intuição de H4 era que pontos Hard, por estarem na fronteira, informariam melhor o LLM sobre a geometria da decisão. Vamos ver se essa intuição se confirma.

Slide 27 — Curvas de Aprendizado

Aqui temos o resultado mais importante da Fase D: as curvas de aprendizado. A tabela mostra a acurácia do LLM versus o expert para cada número de exemplos e cada estratégia.

O ponto de partida é 55.5% em zero-shot — pouco melhor que o aleatório. Com 5 exemplos easy, já sobe para 78.6%. Com 40 exemplos hard, chega a 95.8%.

H3 está claramente confirmada: o LLM consegue aprender o critério do expert, e melhora consistentemente com mais exemplos. A melhoria total é de mais de 40 pontos percentuais.

Mas observem o comportamento da estratégia Hard: com 5 exemplos, fica em 52.5% — pior que zero-shot! Só melhora significativamente a partir de 20 exemplos. Isso é muito diferente de Easy e Random, que já são bons com poucos exemplos.

Slide 28 — Comparação de Estratégias

Agrupando os resultados por estratégia, a média de acurácia para todos os tamanhos de few-shot maiores que zero é: Random com 90.4%, Mixed com 88.1%, Easy com 81.4%, e Hard com 75.3%. H4 é refutada. Exemplos fáceis superam exemplos difíceis consistentemente. E mais: a estratégia aleatória é a melhor! Isso parece contraintuitivo, mas faz sentido quando pensamos em como LLMs processam exemplos in-context.

A interpretação é que exemplos fáceis funcionam como “âncoras” que definem claramente as regiões de cada classe. O LLM consegue interpolar a partir dessas âncoras para classificar pontos intermediários. Exemplos difíceis, por estarem na zona de ambiguidade, não fornecem padrões claros de nenhuma das classes — confundem mais do que ajudam, especialmente com poucos exemplos. Este é um achado que consideramos relevante para a literatura de in-context learning.

Para verificar se os resultados dependem do expert específico, testamos 3 configurações: `aniso_x2` com $W = [0.3, 1.5]$ que é o original, `aniso_x1` com $W = [1.5, 0.3]$ que inverte a importância das dimensões, e `euclidean` com $W = [1.0, 1.0]$ que dá pesos iguais.

Isso responde à pergunta: “O LLM aprende igualmente bem critérios diferentes?”

Como validação, rodamos os 3 algoritmos sobre os rótulos do oráculo — ou seja, sobre as decisões de um classificador que usa exatamente o W do expert. Os 3 algoritmos recuperam o W com similaridade cosseno de 0.99 a 1.00 e fidelidade de 95% a 100%. Isso confirma que os algoritmos estão corretos: quando o classificador é perfeitamente consistente, o W é recuperado com precisão.

O experimento de diluição testa uma preocupação prática: se adicionarmos exemplos fáceis a um conjunto de exemplos difíceis, os fáceis “diluem” a informação dos difíceis?

O design fixa 4 exemplos hard e adiciona progressivamente 0, 2, 4, 10, 16 e 20 exemplos easy. Os resultados mostram que a acurácia melhora monotonicamente: de 56.8% com 4 hard puros para 86.9% com 4 hard + 16 easy. Há uma leve queda para 84.9% com 20 easy, mas não é significativa.

A conclusão é que exemplos fáceis não diluem — eles complementam. Isso reforça o achado de que exemplos fáceis têm valor informativo significativo para o LLM, mesmo na presença de exemplos difíceis.

Slide 31 — Viés de Ordem dos Exemplos

Este é um teste específico da Fase D: a ordem em que os exemplos few-shot são apresentados afeta o resultado? Isso é relevante por causa do chamado “recency bias” — a tendência de LLMs a dar mais peso aos últimos tokens processados.

Testamos 4 ordenações com os mesmos exemplos da estratégia mixed: `class0_first` coloca todos da classe 0 primeiro, `class1_first` inverte, `shuffled` embaralha aleatoriamente, e `alternating` intercala.

O resultado para `seed=42` com 10-shot mostra diferenças pequenas: entre 88.6% e 90.0%. O delta máximo é de 1.4 pontos percentuais, o que é praticamente negligível.

É importante notar que este experimento é diferente do viés de ordem das classes no Slide 21: lá mudamos o texto do prompt, aqui mudamos a ordem dos exemplos fornecidos. O fato de o efeito ser pequeno aqui é reconfortante — sugere que o LLM processa os exemplos de forma relativamente equitativa.

A validação oráculo é nosso “sanity check” principal. A lógica é: se usarmos um classificador sintético perfeitamente consistente no lugar do LLM, os algoritmos deveriam recuperar exatamente o W verdadeiro.

Para o Problema D com o expert `aniso_x2`, o Perceptron recupera W com similaridade cosseno de 0.9999 e fidelidade de 100%. O NNLS fica em 0.9933 e 95.3%. O LP também atinge 0.9999 e 100%. Isso é crucial para a interpretação: quando a fidelidade com o LLM é baixa, sabemos que o problema não está nos algoritmos — está na inconsistência do LLM. Os algoritmos são capazes de resolver o problema quando o input é consistente. A inconsistência medida reflete propriedades reais do LLM, não limitações metodológicas.

O experimento R3 testa se o LLM captura relações não-lineares entre features. A motivação é simular um kernel quadrático sem sair do mundo de métricas lineares: criamos uma terceira feature $x_3 = x_1 \cdot x_2$, que captura a interação entre as duas dimensões originais.

Os mesmos 150 pontos do Problema A são usados, agora com 3 coordenadas no prompt. Os 3 algoritmos aprendem um W com 3 componentes. A pergunta central é: o peso w_3 associado à interação é relevante? O LLM “percebe” a feature de interação?

Também comparamos as decisões do LLM com 2 features versus 3 features: se a concordância for baixa, significa que adicionar a feature extra muda o comportamento do modelo.

Os resultados são interessantes e ambíguos. O Perceptron atribui peso relevante a w_3 — 1.07, comparável a $w_1 = 2.31$ e $w_2 = 1.21$. Isso sugeriria que a interação importa. Mas o NNLS atribui $w_3 \approx 0.03$, quase zero. E o LP distribui igualmente: 0.33 para cada.

A fidelidade R3 é boa: 86% a 90%, melhor que a fidelidade R2. Mas o dado mais revelador é a concordância cruzada: LLM com 2 features versus métrica com 3 features é de apenas 60%. Isso mostra que adicionar x_3 ao prompt realmente muda o comportamento do LLM — ele não ignora a feature extra.

A divergência entre algoritmos sobre w_3 é um resultado em si: mostra que a importância da feature de interação é mal determinada pelo conjunto de dados e depende da formulação do problema de otimização.

Uma pergunta fundamental é: o LLM faz algo que um classificador trivial não faria com os mesmos dados? Para responder, treinamos 3 classificadores clássicos nos exatamente mesmos exemplos few-shot da Fase D.

O k-NN usa k ajustado ao número de exemplos, no máximo 5 e sempre ímpar, com distância euclidiana padrão. A Regressão Logística é linear com regularização L2. E o SVM usa kernel radial gaussiano com parâmetros padrão do sklearn.

A comparação é justa: mesmo treino, mesmo teste, mesmas métricas. A única diferença é o classificador. Se os baselines atingirem performance similar ao LLM, então o LLM não está fazendo nada especial — está simplesmente classificando como qualquer outro algoritmo faria com aqueles dados.

Slide 36 — Baselines Resultados

Os resultados são reveladores. Com exemplos easy e 10-shot, todos os classificadores — incluindo o LLM — ficam entre 88% e 90%. Os pontos são trivialmente separáveis e qualquer método funciona. Mas com exemplos hard, a história é diferente. O k-NN despenca para 62%, o SVM para 48%, mas a Regressão Logística atinge 100%! Isso faz sentido: a fronteira do expert é essencialmente linear, e a LR aprende exatamente essa fronteira. O k-NN e o SVM-RBF sofrem porque os poucos pontos na fronteira não fornecem vizinhança suficiente.

E o LLM? Fica em 68.5% com hard — melhor que k-NN e SVM, mas muito abaixo da LR. A conclusão é que o LLM não é uniformemente superior. Sua vantagem aparece em cenários de poucos exemplos ambíguos onde k-NN e SVM falham, mas a LR demonstra que algoritmos clássicos com a indução correta — viés linear — podem ser superiores.

Olhando a robustez global entre as 3 seeds, a consistência média no Problema B é 81.1% com desvio de 14.8%, e no Problema C é 77.9% com desvio de 16.8%. A variabilidade é alta, especialmente em zero-shot.

A boa notícia é que os resultados few-shot são consideravelmente mais estáveis: o desvio cai significativamente com mais exemplos. Isso reforça que o LLM, quando orientado, é mais previsível. A instabilidade de W entre seeds, já discutida anteriormente, é a principal fragilidade de H5. Porém, é importante notar que a instabilidade de W não necessariamente implica instabilidade das classificações — métricas diferentes podem produzir classificações semelhantes em certas geometrias.

Para rigor estatístico, usamos Bootstrap com 10.000 reamostras para calcular intervalos de confiança de 95%.

O teste pareado entre zero-shot e 10-shot mostra que a melhoria é estatisticamente significativa: delta de +0.322 no Problema B com intervalo $[+0.257, +0.360]$ que exclui zero, e delta de +0.373 no Problema C com intervalo $[+0.342, +0.400]$. Os tamanhos de efeito, medidos por Cohen's d, são de 7.08 e 19.81 — efeitos extremamente grandes.

Na Fase D, o teste Easy vs. Hard com 5-shot mostra delta de +0.231 com CI $[+0.100, +0.340]$, confirmando estatisticamente que Easy supera Hard nos tamanhos menores de few-shot.

O teste da métrica estimada versus a euclidiana em zero-shot mostra diferença não significativa: CI inclui zero. Isso significa que, em zero-shot, a métrica aprendida não é significativamente melhor que a métrica euclidiana trivial.

Vamos à síntese final das hipóteses. H1 sobre consistência é parcialmente confirmada: Kappa vai de 0.14 em zero-shot para 0.93 com 10 exemplos. Consistência existe, mas depende de orientação.

H2 sobre few-shot é fortemente confirmada: o salto de 30 pontos percentuais com apenas 5 exemplos é um efeito robusto e reproduzível.

H3 sobre o LLM como aprendiz está confirmada: o LLM vai de 55% a 96% com exemplos do expert.

H4 sobre exemplos difíceis é refutada — e consideramos que esta refutação é um dos achados mais valiosos do trabalho. LLMs se beneficiam mais de exemplos prototípicos que de exemplos fronteiriços.

H5 sobre estabilidade é parcial: W é instável entre seeds, mas classificações few-shot são estáveis.

As contribuições deste trabalho são quatro. Primeira: propomos um framework de otimização inversa aplicado a LLMs, com 3 algoritmos independentes validados por oráculo. Esta é uma abordagem nova que conecta a área de metric learning com a análise de decisões de modelos de linguagem.

Segunda: fornecemos evidência empírica sobre a consistência decisória de LLMs — consistentes com orientação, inconsistentes sem.

Terceira: a refutação de H4 é um achado valioso para a comunidade de in-context learning. Mostramos que exemplos prototípicos são mais eficazes que exemplos fronteirços para LLMs, o que contrasta com a intuição clássica de active learning.

Quarta: realizamos uma análise de sensibilidade abrangente que identifica confundidores — viés de classe, semântica de features, template de prompt, ordem de exemplos — que devem ser controlados em trabalhos futuros.

Slide 41 — Limitações e Trabalhos Futuros

Quanto às limitações, a mais importante é que testamos apenas um modelo, o gpt-4o-mini. Não sabemos se os achados se generalizam para Claude, Gemini, ou GPT-4o. Segundo, usamos apenas dados sintéticos 2D — dados reais e alta dimensão podem se comportar diferentemente. Terceiro, a métrica diagonal com $O(d)$ parâmetros é uma simplificação que pode não capturar interações entre features. A temperatura fixa em 0.0 também limita a generalização. E o LP frequentemente retornou infeasible, o que limita a utilidade desse algoritmo.

Para trabalhos futuros, os caminhos mais promissores são: testar múltiplos LLMs para separar propriedades de LLMs em geral de propriedades do gpt-4o-mini especificamente; usar dados reais como Iris e MNIST; explorar a Mahalanobis completa; investigar fronteiras não-lineares; e, talvez o mais impactante, aplicar esta abordagem para auditoria prática de decisões de IA em domínios críticos como saúde e justiça.

Gostaria de agradecer ao meu orientador pela paciência e pelas contribuições fundamentais ao direcionamento da pesquisa, à banca por aceitar avaliar este trabalho, e a todos que acompanharam esta jornada.

Fico à disposição para perguntas. Estou preparado para discutir qualquer aspecto da metodologia, dos resultados ou das implicações do trabalho.

Obrigado.